

Práctica Next.js – Explorador de Pokémon

[Nome do estudante / Parella]

Objetivo

Desarrollar una aplicación web con **Next.js** que consuma la API pública **PokéAPI** y permita:

- Listar todos los Pokémon en una página principal.
- Buscar Pokémon por nombre en tiempo real.
- Navegar a una página dinámica con el detalle de cada Pokémon.
- Volver a la página principal desde la vista de detalle.

La práctica tiene como objetivo trabajar:

- Enrutado estático y dinámico en Next.js.
- Componentización.
- Fetching de datos desde una API REST.
- Manejo de estado.
- Props.
- Navegación con `Link` y/o `useRouter`.
- Estilado con CSS clásico.

API a utilizar

PokéAPI

Endpoint base recomendado para listar Pokémon con su nombre y imagen:

`https://pokeapi.co/api/v2/pokemon?limit=151`

Para obtener detalles de cada Pokémon:

`https://pokeapi.co/api/v2/pokemon/{nombre}`

Requisitos funcionales

1 Página Principal (/)

Debe:

- Mostrar todos los Pokémon obtenidos desde la API.
- Renderizar cada Pokémon mediante un componente reusable `<PokemonCard />`.
- Cada tarjeta debe mostrar:
 - Imagen del Pokémon.
 - Nombre.
- Incluir un input de búsqueda que:
 - Permita filtrar Pokémon por nombre.
 - Filtre en tiempo real (sin recargar la página).

2 Componente PokemonCard

Debe:

- Recibir por `props` los datos del Pokémon.
- Mostrar imagen y nombre.
- Ser clicable.
- Redirigir a la ruta dinámica correspondiente al Pokémon.

Ejemplo de ruta esperada:

```
/pokemon/pikachu  
/pokemon/charmander  
/pokemon/squirtle
```

3 Página dinámica de Pokémon

Ruta dinámica obligatoria:

```
/pokemon/[name]
```

Debe:

- Obtener la información del Pokémon seleccionado.
- Mostrar una página completa con al menos:
 - Nombre
 - Imagen
 - Tipos
 - Habilidades
 - Peso y altura
- Incluir un botón o enlace visible para volver a la página principal.

Estilado

- Se debe usar CSS clásico (CSS Modules o archivo global).
- Se valorará:
 - Diseño limpio y legible.
 - Buen uso de espaciado.
 - Responsive básico.
 - Efectos hover en tarjetas.
 - Jerarquía visual clara.

Requisitos técnicos

- Proyecto creado con `create-next-app`.
- Uso de rutas dinámicas.
- Componentes reutilizables.
- Uso correcto de `Link` o `useRouter`.
- Manejo correcto de estado para el buscador.
- Código limpio y organizado.
- Estructura de carpetas coherente.

Entrega

- Subir el proyecto a **GitHub**.
- Incluir un `README.md` con:
 - Instrucciones para ejecutar el proyecto.
 - Breve explicación de la estructura.
 - Miembros participantes si se hace en pareja.

Evaluación

Se valorará especialmente:

- Correcto funcionamiento de las rutas dinámicas.
- Calidad del código.
- Componentización.
- Buen uso de Next.js.

- Experiencia de usuario.
- Diseño y CSS.
- Limpieza del repositorio.